```
    if len(replay) == replay_size:                          9
        indx =
np.random.randint(low=0,high=len(replay),size=batch_size)
        exps = [replay[j] for j in indx]
        state_batch = torch.stack([ex[0] for ex in
exps],dim=1).squeeze()
        action_batch = torch.Tensor([ex[1] for ex in exps])
        reward_batch = torch.Tensor([ex[2] for ex in exps])
        state2_batch = torch.stack([ex[3] for ex in
exps],dim=1).squeeze()
        pred_batch =
dist_dqn(state_batch.detach(),theta,aspace=aspace)
        pred2_batch =
dist_dqn(state2_batch.detach(),theta_2,aspace=aspace)
        target_dist =
get_target_dist(pred2_batch,action_batch,reward_batch, \
                                    support, lim=
(vmin,vmax),gamma=gamma)
        loss = lossfn(pred_batch,target_dist.detach())
        losses.append(loss.item())
        loss.backward()
        with torch.no_grad():                               10
            theta -= lr * theta.grad
        theta.requires_grad = True

    if i % update_freq == 0:                                11
        theta_2 = theta.detach().clone()

    if i > 100 and eps > eps_min:                           12
        dec = 1./np.log2(i)
        dec /= 1e3
        eps -= dec

    if done:                                                13
        state = preproc_state(env.reset())
        done = False
```

- ***1*** Epsilon-greedy action selection
- ***2*** Takes selected action in the environment
- ***3*** Changes reward to +10 if environment produced reward of 1 (successful freeway crossing)
- ***4*** Changes reward to –10 if game is over (no crossings after a long time)
- ***5*** Changes reward to –1 if original reward was 0 (game is just continuing) to penalize doing nothing
- ***6*** Prepares experience as a tuple of the starting state, the observed reward, the action taken, and the subsequent state